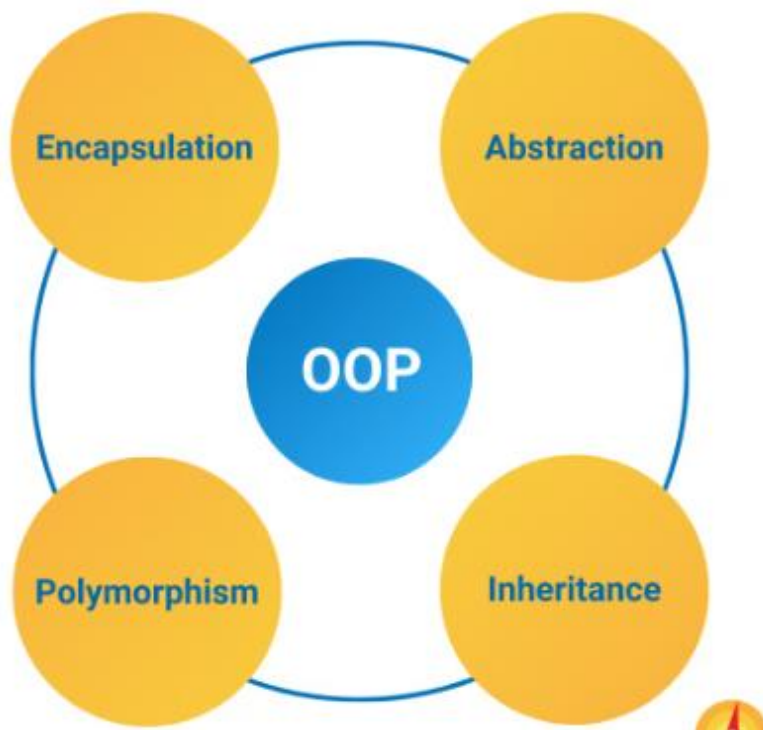


Object-Oriented (OO) development

Object-Oriented (OO) development is a programming paradigm that organizes code into objects, which are instances of classes that encapsulate data (attributes) and behavior (methods). It's built on core principles like encapsulation, inheritance, polymorphism, and abstraction, which make software design more modular, reusable, and maintainable.



Key Principles of OO Development

1. **Encapsulation:** Bundling data and methods that operate on that data within a class, restricting direct access to some components (e.g., using private/protected modifiers). This protects the object's internal state and exposes only necessary interfaces.

```
// Encapsulation: Base class to store and protect person's details
abstract class Person {
    private String name;        // Private: Person's name
    private int leaveBalance;    // Private: Available leave days

    public Person(String name, int leaveBalance) {
        this.name = name;
        this.leaveBalance = leaveBalance;
    }

    public String getName() {
        return name;
    }

    public int getLeaveBalance() {
        return leaveBalance;
    }

    protected boolean useLeave(int days) {
        if (days <= leaveBalance) {
            leaveBalance -= days;
            return true;
        }
        return false;
    }
}
```

(The Person class is like a locked box that holds a person's name and leaveBalance. These are private, meaning no one can touch them directly. You can only use methods like getName() or useLeave() to check or change them, like asking a teacher for permission.)

2. **Inheritance:** Allows a class (subclass) to inherit properties and methods from another class (superclass), promoting code reuse and hierarchical relationships (e.g., a Dog class inheriting from an Animal class).

(Employee and Student are types of Person, so they get the name and leaveBalance features for free. It's like how both dogs and cats are animals

and share traits like having legs. Employee adds an employeeId, and Student adds a studentGrade.)

3. **Polymorphism:** Enables objects to be treated as instances of their parent class while retaining their unique behavior. For example, a Shape class can have a draw() method overridden by Circle or Rectangle subclasses.

(Both Employee and Student can request leave, but they do it differently. When requestLeave is called, an Employee might say, "Employee Alice got leave!" while a Student says, "Student Bob got leave!" It's like how dogs bark and cats meow, but both make a sound.)

```
// Inheritance and Polymorphism: Employee inherits from Person and implements LeaveRequest
class Employee extends Person implements LeaveRequest {
    private String employeeId; // Unique for Employee

    public Employee(String name, int leaveBalance, String employeeId) {
        super(name, leaveBalance);
        this.employeeId = employeeId;
    }

    // Polymorphism: Employee's way of requesting leave
    @Override
    public String requestLeave(int days) {
        if (useLeave(days)) {
            return "Employee " + getName() + " (ID: " + employeeId + ") got " + days + " days leave approved!";
        }
        return "Employee " + getName() + " does not have enough leave days.";
    }
}
```

```
// Inheritance and Polymorphism: Student inherits from Person and implements LeaveRequest
class Student extends Person implements LeaveRequest {
    private String studentGrade; // Unique for Student

    public Student(String name, int leaveBalance, String studentGrade) {
        super(name, leaveBalance);
        this.studentGrade = studentGrade;
    }

    // Polymorphism: Student's way of requesting leave
    @Override
    public String requestLeave(int days) {
        if (useLeave(days)) {
            return "Student " + getName() + " (Grade: " + studentGrade + ") got " + days + " days leave approved!";
        }
        return "Student " + getName() + " does not have enough leave days.";
    }
}
}
```

4. **Abstraction:** Hides complex implementation details and exposes only simplified interfaces, making systems easier to use and understand (e.g., a Car class abstracting engine mechanics into a drive() method).

(The LeaveRequest interface is like a rule that says, “Anyone who wants to request leave must have a requestLeave method.” It doesn’t care how it’s done, just that it’s done. This keeps things simple, like pressing a button on a toy without knowing how it works inside.)

```
// Abstraction: Interface defining the contract for leave requests
interface LeaveRequest {
    String requestLeave(int days);
}
```

```
// Main class to demonstrate the system
public class LeaveManagement {
    public static void processLeave(LeaveRequest leaveTaker, int days) {
        System.out.println(leaveTaker.requestLeave(days));
    }

    public static void main(String[] args) {
        Employee employee = new Employee("Alice", 10, "E123");
        Student student = new Student("Bob", 5, "8th Grade");

        processLeave(employee, 3); // Employee requests 3 days
        processLeave(student, 2); // Student requests 2 days
        processLeave(employee, 8); // Employee tries too many days
    }
}
```